1. Introduction

Nim Game

Nim is an old two-player game. Several piles of objects are laid out on the table and two players take turns. On your turn, you pick one pile and take away at least one object from it — you may take the whole pile if you want. There is only one restriction: each move touches a single pile. The player who takes the last object wins

Halving Game

The Halving game generates a number from 15 to 50 randomly. Two players take turns to operate. In each turn, one of the players can either choose to remove a stone or remove half the amount of the stones (integer division). Minimax fits for halving games because it has a limited amount of game state. Each turn can create a new game state. Players can be able to observe the whole game state.

Connect Four

Connect Four is a two-player game, while we are using a compact 4-row, 4-column (4x4) variant here. Players drop tokens into any non-full column; pieces fall to the lowest empty cell in that column. A player wins by forming an unbroken horizontal, vertical, or diagonal line of four identical tokens. The game terminates immediately upon a win, or ends as a tie if all 16 cells fill with no four-in-a-row.

Tic-Tac-Toe

Tic-Tac-Toe is a two-player game on a 3×3 board. Players O and X take turns putting a mark in an empty space. A player wins by getting three marks in the same row, column, or diagonal. If the board is full and nobody wins, the game ends in a tie.

Tic-Tac-Toe is good for minimax because the rules are simple and the number of possible states is limited. Both players can see the whole board, and there are no random events.

Why these games are good fit for minimax

- **Turn-based and zero-sum**: if you win, your opponent loses.
- **Always finishes in finitely many moves:** every move brings the game closer to the end. Objects are removed in Nim and Halving, while empty spaces are filled in Tic-Tac-Toe and Connect Four. Therefore, the games cannot continue forever.

2. Strategy and Implementation

Shared Game engine

All four games in this assignment (Nim, Connect Four,Tic-Tac-Toe, and the Halving Game) are built on the same small engine. It is an interface called Game, defined in game.py, that asks every game to provide seven methods. Each game is then a class that fills those seven methods according to its own rules. The engine itself knows no strategy — it only knows how to play by the rules. The strategies (minimax, random, greedy) are written separately and work on top of the engine, so the same strategy code can play any game that follows this interface.

• start_state() — the opening state.
• player(state) — whose turn it is right now (0 or 1).
• actions(state) — every legal move from the current state.
• next(state, action) — the state that results from applying one move (and the turn passes to the other player).
• successors(state) — returns all states that can be reached after one legal move.
• is_end(state) — whether the game is over.
• utility(state) — the final result once the game is over.

Once a game fills these in, minimax can search its game tree without knowing anything about the game's specific rules.

Nim-minimax

Strategy

It is written as a function with the signature minimax(game, state, remember):

• game is the Game object, used to look up the rules;

• state is the current state (player, heaps);

• remember is a dictionary for caching (explained below).

The function returns the winner's number (0 or 1) for the given state.

Look at every legal move from this state. For each one, recurse on the state it leads to and find out who wins that branch. The current player wants to win, so as soon as it finds one move that leads to itself winning, it stops and returns its own number — that move is a winning move, no need to look further. If no such move exists, every move leads to the opponent winning, so the current player loses and the function returns the opponent's number.

Enhancement

Pure recursion has a problem: the same table can be reached through many different paths. So we added a dictionary called `remember`. Every time we compute the win/loss for a table, we store it, and the next time we see the same table we just look it up instead of recomputing.

Connect Four-minimax

Strategy

It is written as a function with the signature minimax(game, state, depth_limit):

- **game** is the Game object, used to look up the rules;
- **state** is the current state (board, current player);
- **depth_limit** is an integer to control the maximum recursion depth and avoid infinite search.

The function returns a numeric utility score for the given state, representing the goodness of the board for the first player.
It looks at every legal move from this state. For each one, it recurses on the state it leads to and evaluates the utility of that branch. The maximizer (first player) wants to maximize the state score, so it picks the move that yields the highest utility value. The minimizer (second player) wants to minimize the state score, so it picks the move that yields the lowest utility value. If the state is terminal, it directly returns the predefined utility value (+1 for win, -1 for loss, 0 for tie). If no depth remains, recursion stops and returns a neutral score.

Enhancement

Pure unlimited recursion has a problem: the search tree can become extremely large and cause slow computation. So we added a **depth limit** enhancement. The algorithm only searches up to 4 layers deep, which effectively reduces redundant calculation and speeds up simulation while retaining valid short-term threat prediction.

Tic-Tac-Toe Minimax

The Tic-Tac-Toe state is represented as a tuple containing a 3×3 nested list and the current player. Empty cells contain an empty string, while occupied cells contain `"O"` or `"X"`. An action is represented by a `(row, column)` tuple.

Terminal states use the utility values +1 for an O win, -1 for an X win, and 0 for a tie. The minimax function recursively evaluates every legal successor. On O's turn it returns the maximum successor score, while on X's turn it returns the minimum score. The action function evaluates every available position and selects the action with the best minimax score.

Because the Tic-Tac-Toe search space is small, the agent searches to terminal states without a depth limit.

3. Simulation Design

Nim

Data

We ran **1200 matches** in total. The breakdown is:

- Two opponents: Random and Greedy.
- Five starting tables: `(3,4,5)`, `(1,2,3)`, `(7,7)`, `(1,4,5,6)`, `(2,3,4,5,6)`.
- 120 matches per starting table, with minimax playing first in half of them and second in the other half (alternating),

Data record and store

Every match is written as one row recording: the opponent, the starting table, whether minimax moved first, who won, and how many moves the game lasted. All 1200 rows go into `results.csv`.The data is processed with pandas: read the csv, group by opponent and by turn order, count how many games minimax won, divide by the group size to get the win rate, and plot the results.

Halving game

The test objects are two agents. The minimax_bot will adopt a minimax algorithm to choose the best action according to future state. The random_bot will choose an action randomly without considering anything. It will execute 100 times automatically. Then, it will count how many times each player wins. The simulation data is stored as a .json file.

Connect Four

Data

The test objects are two agents. The minimax bot will adopt a minimax algorithm to choose the best action according to future game states. The random bot will choose an action randomly without considering game strategy. We ran 100 matches in total. Minimax always plays first, and the random agent plays second, starting from an empty 4×4 board every time.

Data record and store

Every match records the winner and total game steps. All simulation records and statistical results are automatically stored in a .json file. After simulation, the program calculates win counts, draw counts, win rate and average game steps for result analysis and visualization.

Tic-Tac-Toe

We simulated 500 matches between a RandomBot and a MinimaxBot. RandomBot played first as O, while MinimaxBot played second as X. Every match started with an empty 3×3 board.

Each match was stored as one row in `tictactoe_results.csv`. The recorded fields were the O bot, the X bot, the winner, and the number of moves. Pandas was used to read the CSV, count each result, calculate game-length statistics, and generate a bar chart.

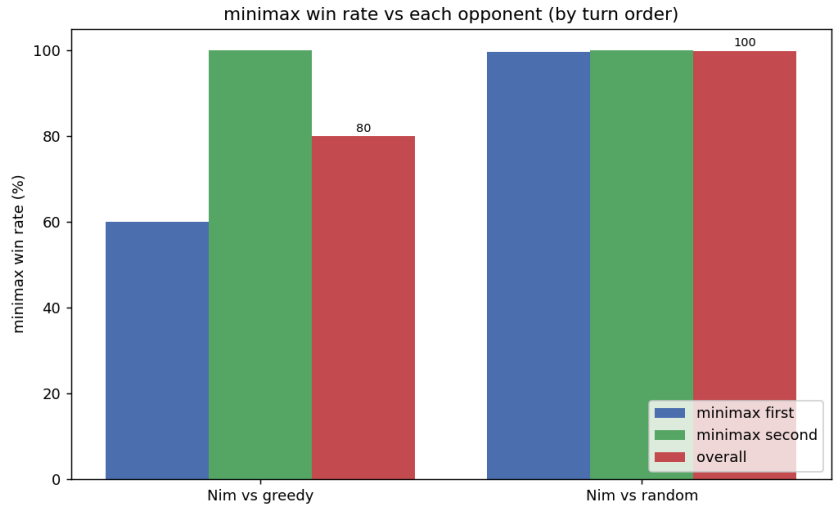
4. Results Discussion

Nim

Against Random, minimax wins almost every game (99.8%).

Winning 100% as the second player. The weak opponent moves first and almost always makes a mistake, turning what should have been a winning table into a losing one.

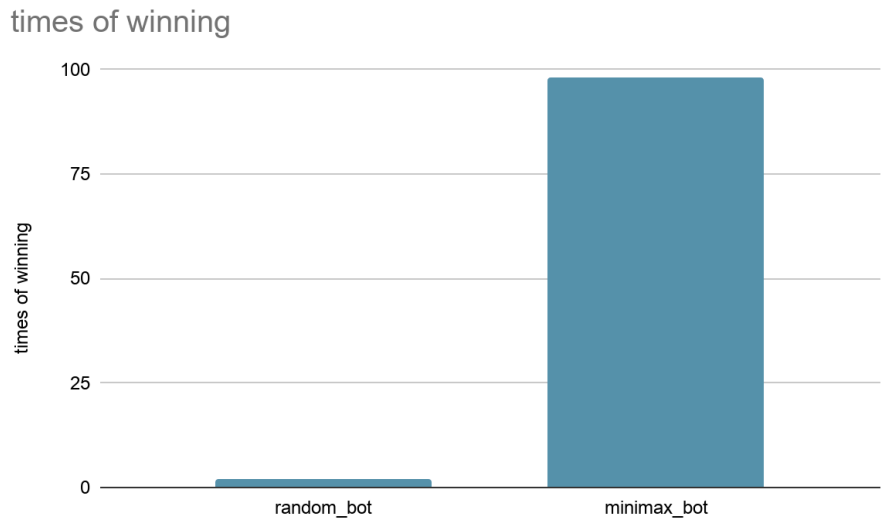
Only 60% as first player against Greedy. Among the five starting tables, two of them — `(1,2,3)` and `(7,7)` — are so-called losing starts. From such a table, **whoever moves first is at a disadvantage**, because every move hands the winning position to the opponent.



Halving Game

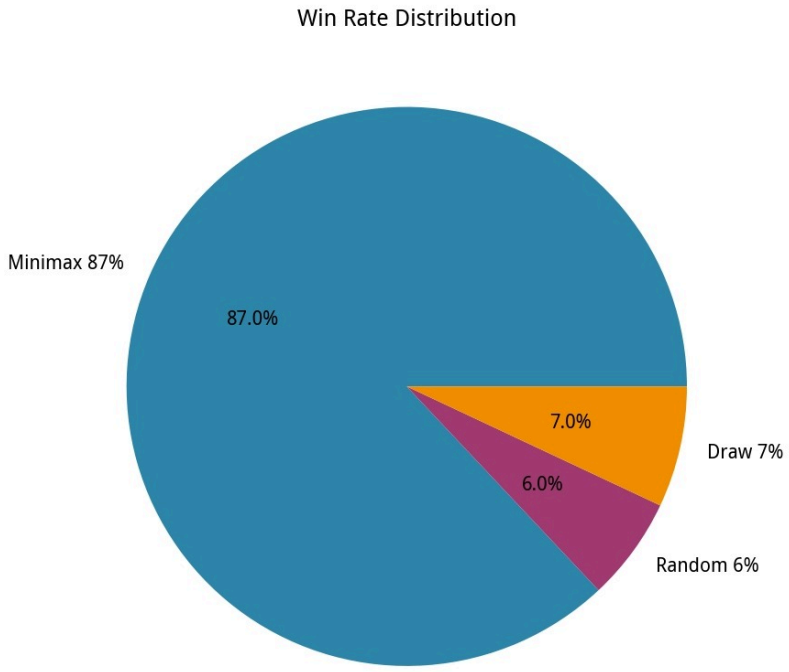
This game simulated it 100 times. The results are minimax_bot wins 98 times and the random_bot wins 2 times only. Therefore, the result shows that minimax_bot has an advantage. Even though sometimes it loses a few games, because the random_bot maybe has a good start state.

This experiment shows that the minimax algorithm has a better strategy compared to a normal agent.



Connect Four

This game simulated 100 times. The results are minimax bot wins 87 times, random bot wins 6 times, and 7 games end in ties. Therefore, the result shows that the minimax bot has a huge advantage over the random agent.
The random agent only wins occasionally by accidental token connections, with no strategic planning or defensive blocking. Minimax consistently predicts future states, builds winning lines and blocks potential threats, leading to an extremely high winning rate.
This experiment shows that the minimax algorithm has far better strategic decision-making ability compared with normal random agents in perfect-information zero-sum games.



Tic-Tac-Toe

The MinimaxBot won 397 of the 500 matches (79.4%), while 103 matches (20.6%) ended in ties. RandomBot won no matches. The average game length was 7.374 moves.

These results show that Minimax never loses against the random policy, even when Minimax plays second. It wins when RandomBot makes a move that creates a forced winning path and produces a tie when RandomBot happens to follow a safe sequence. This matches the theoretical expectation that a perfect Tic-Tac-Toe player can always avoid defeat.

